

M347 Examen : Déploiement avancé d'une Application PHP/MariaDB avec Docker Compose

Objectifs

Cet examen vise à évaluer votre capacité à :

- Conteneuriser une application web PHP/MariaDB en appliquant les meilleures pratiques avancées vues dans le cours M347.
- Concevoir une architecture réseau Docker évolutive, claire et sécurisée.
- Assurer la résilience, la maintenabilité et la sécurité d'un déploiement Docker complexe.

Ressources fournies

- Code source PHP complet d'une nouvelle application (`php-app/src`).
- Script SQL (`init_mariadb.sql`) pour initialiser une base de données MariaDB.
- Docker Desktop (version stable actuelle – vérifiez avant l'examen).

Interdictions et restrictions

- **Strictement interdit d'utiliser l'IA** pour générer du code ou des configurations.
- **Internet limité aux seuls accès suivants :**
 - **Moodle** : ressources et dépôt.
 - **Docker Hub** : consultation des images officielles uniquement.

Structure du code fourni

```
docker-php-mariadb
├── db
│   └── scripts
│       └── init_mariadb.sql
└── php-app
    ├── src
    │   ├── assets
    │   │   └── style.css
    │   ├── pages
    │   │   ├── accueil.php
    │   │   ├── contact.php
    │   │   └── about.php
    └── index.php
```

Consignes précises

Votre travail consiste à :

1. Créer des Dockerfiles optimisés :

- Appliquez les principes d'optimisation des Dockerfiles : regroupement de commandes, choix d'images légères officielles et nettoyage immédiat.

2. Configurer le fichier docker-compose.yml :

- Définissez explicitement les services suivants :
 - **php-app** : Application PHP (front + back actuels).
 - **mariadb** : Base MariaDB (image officielle, version 11).
- Définissez deux réseaux Docker distincts :
 - **front_net** : réseau d'accès utilisateur à l'application PHP.
 - **back_net** : réseau interne pour les communications PHP <-> MariaDB, isolé de l'extérieur.
- Configurez précisément les volumes :
 - Code PHP (php-app/src) accessible en temps réel.
 - Stockage persistant des données MariaDB pour préserver la continuité des données au-delà du conteneur.
- Configurez précisément les mécanismes de résilience (restart: always , depends_on , healthcheck) pour les deux services afin d'assurer robustesse et disponibilité.

3. Sécuriser la gestion des informations sensibles :

- Créez et utilisez un fichier `.env` pour stocker toutes les variables sensibles (utilisateur, mot de passe, base de données, etc.), cf :

```
MYSQL_ROOT_PASSWORD=root_secret
MYSQL_DATABASE=site_db
MYSQL_USER=app_user
MYSQL_PASSWORD=app_pass
```

```
DB_HOST=mariadb
DB_NAME=site_db
DB_USER=app_user
DB_PASSWORD=app_pass
```

- Intégrez ces variables dans votre `docker-compose.yml`.

4. Tests obligatoires à effectuer :

- Vérifiez que l'application PHP fonctionne correctement et est accessible (tests en ligne de commande ou fonctionnels sur le navigateur).
- Vérifiez **en ligne de commande depuis votre terminal** que le conteneur de la base de données MariaDB est correctement initialisé via le script d'initialisation.

5. Preuves à fournir :

- Enregistrez toutes les commandes Docker exécutées pendant l'examen dans un fichier `historique_commandes.txt` :
 - Bash :

```
history | grep docker > historique_commandes.txt
```

- PowerShell :

```
Get-History
| Where-Object CommandLine -Like "*docker*"
| Select-Object -ExpandProperty CommandLine
| Out-File historique_commandes.txt
```

Critères d'évaluation

- Qualité technique et optimisation des Dockerfiles.

- Précision et justification des choix techniques dans le docker-compose.
- Sécurité et robustesse démontrées par des tests explicites.

Livrables sur Moodle

- Un .zip contenant :
 - exactement la structure des fichiers demandée ci-dessus.
 - Un fichier texte nommé `historique_commandes.txt` avec toutes les commandes Docker exécutées durant l'examen.
 - y compris les commandes de test du conteneur MariaDB.

Bonne chance à tous !